Python II module - Exercises

Pieter Kleer

Table of contents

Overview 2

Lecture 1: Sections 3.1-3.5 3

Lecture 2: Sections 3.6-3.7 and 4.1-4.3 6

Overview

This online document contains all the exercises corresponding to the lectures. Solutions are provided on a week-by-week basis in a different document.

Lecture 1: Sections 3.1-3.5

Make sure to import NumPy to be able to use all its functionality.

```
import numpy as np
```

Do not use for- or while-loops when answering the questions below.

Question 1

Create a 3×2 array M of ones, i.e., the matrix

```
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```

Question 2

Create the array $x = [2, 4, 6, 8, ..., 100]$ once with `np.arange()`, and once with `np.linspace()`.

Question 3

Create the array $x = [-1, -0.9, \dots, -0.1, 0, 0.1, \dots, 0.9, 1]$.

Question 4

Consider the two-dimensional array M and answer the questions below. Recall that Python always starts counting at 0 when indexing.

```
M = np.array([[1,1,1,1,1],[2,1,2,1,2],[2,1,2,1,2],
              [2,1,2,1,2],[2,1,2,1,2],[1,1,1,1,1]])

print(M)
```

```
[[1 1 1 1 1]
 [2 1 2 1 2]
 [2 1 2 1 2]
 [2 1 2 1 2]
 [2 1 2 1 2]
 [1 1 1 1 1]]
```

a) Access the rows of M with odd indices $1, 3$ and 5. Your output should be

```
[[2 1 2 1 2]
 [2 1 2 1 2]
 [1 1 1 1 1]]
```

b) Index/access the rows and columns that form the submatrix consisting of elements equal to 2, i.e., obtain from M the matrix

```
[[2 2 2]
 [2 2 2]
 [2 2 2]
 [2 2 2]]
```

c) Access the submatrix consisting of the rows with indices $0, 1$ and 5 and columns with indices $1, 3, 4$, i.e.,

```
[[1 1 1]
 [1 1 2]
 [1 1 1]]
```

Question 5

Consider the following array.

```
x = np.array([3,6,4,5,5,5,1,4,2,9,6,7,11,10])

print(x)
```

```
[ 3  6  4  5  5  5  1  4  2  9  6  7 11 10]
```

a) Use a Boolean mask to access all elements whose value is less than or equal to 4.

b) Use a Boolean mask to access all elements whose value is in the interval $[5, 10]$.

c) Use a Boolean mask to access all elements whose value is in the interval $[2, 4]$ or $[6, 9]$.

Question 6

Take the array $x = [0, 1, 2, 3]$ and convert it to

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 & 0 & 1 & 2 & 3 \end{bmatrix}.$$

Question 7

Construct the following matrix M using appropriate NumPy functions starting from the array $[1, 2, 4]$:

$$\begin{bmatrix} 1 & 1 & 2 & 2 & 4 & 4 \\ 1 & 1 & 2 & 2 & 4 & 4 \end{bmatrix}.$$

Question 8

Write a function `blocks(m,n)` that, for given integer inputs m and n, returns an $(m+n) \times (m+n)$ matrix that contains an $m \times m$ block of ones on the top left, and an $n \times n$ block of ones on the bottom right (and zeros elsewhere). Use `hstack()` and `vstack()` in your solution.

For $m = 2$ and $n = 3$, this results in the matrix

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Question 9

Write a function `checkerboard(n)` that returns for an integer n a checkerboard pattern of zeros and ones of size $n \times n$ (see examples below; the top-left element is always a 1).

For $n = 5$ and $n = 6$, the matrices should look as follows:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Question 10

Compute the anti-diagonal of the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & -7 \\ -9 & 10 & 11 \end{bmatrix}$$

using `np.rot90()`. The anti-diagonal is obtained by going from the bottom-left to the top-right element, i.e., $[-9, 6, 3]$ in this case (and not $[3, 6, -9]$). Have a look at the documentation of `np.rot90()` for details of how this function works.

Lecture 2: Sections 3.6-3.7 and 4.1-4.3

Make sure to import NumPy to be able to use all its functionality. We also add a command that displays floating-point numbers in arrays to three decimal places. You do not have to know this command.

```
import numpy as np
np.set_printoptions(precision=3, floatmode="fixed")
```

Do not use for- or while-loops when answering the questions below.

Question 1

Create the two-dimensional array

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \end{bmatrix}$$

by combining two functions seen in Chapter 3.

Question 2

Create the array

$$x = [1, 5, 9, 2, 6, 10, 3, 7, 11, 4, 8, 12]$$

from the first three rows of M (using reshaping functionality in combination with transposing a matrix).

Question 3

Implement the following function

$$f(x) = \begin{cases} -x + 3 & \text{if } x < 0 \\ x^2 + 3 & \text{if } 0 \leq x < 1 \\ \sqrt{x^2 + 3} + 2 & \text{if } x \geq 1 \end{cases}$$

so that it can handle both single numbers x and one-dimensional arrays $x \in \mathbb{R}^n$ as input. You might want to look at the Heaviside function example in Section 4.1 for some inspiration.

Your function should give the following output on the given input x.

```
x = np.arange(-5,6)
print(f(x))
```

```
[8.000 7.000 6.000 5.000 4.000 3.000 4.000 4.646 5.464 6.359 7.292]
```

Question 4

We will write a function that can compute the cumulative mean of a one-dimensional array. Take $n = 10$ (define this as a variable in your script).

a) Create the array $y = [1, 1/2, 1/3, \dots, 1/(n-1), 1/n]$ using `np.arange()` in combination with a division.

b) By combining your solution in part a) with `np.cumsum()`, create a function `cum_mean()` that takes as input a one-dimensional array $x = [x_0, \dots, x_{n-1}]$ and outputs the cumulative means of the array. This is the array that has at position i the value

$$\frac{1}{i+1} \sum_{j=0}^{i} x_j$$

for $i = 0, \dots, n-1$.

It should give the following output on the given input x.

```
# Some test data
x = np.array([1,4,2,5])
print(cum_mean(x))
```

```
[1.000 2.500 2.333 3.000]
```

c) Vectorize your function from part b) so that it takes as input two-dimensional arrays, and outputs the cumulative mean of every row of the two-dimensional array. Your function should give the following output on the given input matrix M.

```
# Some test data
M = np.array([[1,4,2,5],[1,10,12,8],[-1,9,3,-10]])
print(cum_mean(M))
```

```
[[ 1.000  2.500  2.333  3.000]
 [ 1.000  5.500  7.667  7.750]
 [-1.000  4.000  3.667  0.250]]
```

Question 5

Consider the following function

$$g(x_0, \dots, x_{n-1}) = \sum_{i=0}^{n-1} \sin(x_i) \cdot (x_i)^{2 \cdot i}$$

that takes as input an array $x = [x_0, \dots, x_{n-1}] \in \mathbb{R}^n$ and outputs $g(x)$.

a) Implement the function g. It should give the following output on the given input x.

```
# Some input data
x = np.array([1,4,2,6,4,5])
print(g(x))
```

```
-9427125.80618379
```

b) Vectorize the function g so that it can take as input a two-dimensional array, and return the function value $g(x)$ for every row x of the array.

It should give the following output on the given input M.

```
# Some input data
M = np.array([[1,4,2,6,4,5],[1,4,2,6,4,5],[7,4,9,6,3,5]])
print(g(M))
```

```
[-9427125.806 -9427125.806 -9373912.933]
```

Question 6

Write a function `geom(x)` that takes as input a two-dimensional array, and outputs the geometric mean of every column of the array. For an array $x = [x_0, \dots, x_{n-1}] \in \mathbb{R}^n_{\geq 0}$, the geometric mean is defined as

$$\left(\prod_{i=0}^{n-1} x_i \right)^{1/n}$$

It should give the following output on the given input M.

```
# Some input data
M = np.array([[1,4,2,6,4,5],[1,4,3,7,1,5],[1,4,2,6,8,50]])
print(geom(M))
```

```
[ 1.000  4.000  2.289  6.316  3.175 10.772]
```

Question 7

In this exercise we will normalize the data in an array, so that all entries are between 0 and 1.

a) Write a function `normalize()` that normalizes an array $x = [x_0, \dots, x_{n-1}]$ by replacing every entry x_i by

$$\frac{x_i - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

where $x_{\text{min}} = \min_i x_i$ and $x_{\text{max}} = \max_i x_i$. You may assume that $x_{\text{max}} \neq x_{\text{min}}$ to avoid dividing by zero. *You could add a check for this case into your function, but is not needed.*

It should give the following output on the given input x.

```
# Some input data
x = np.array([1,4,2,-6,4,5])
print(normalize(x))
```

```
[0.636 0.909 0.727 0.000 0.909 1.000]
```

b) Vectorize your function so that it can normalize every column of a two-dimensional array using the formula in part a). You may, again, assume that the maximum and minimum element in each column are different.

It should give the following output on the given input M.

```
# Some input data
M = np.array([[1,4,2,-6,4,5],[-4,3,5,1,3,2],[9,8,7,6,5,4]])
print(normalize(M))
```

```
[[0.385 0.200 0.000 0.000 0.500 1.000]
 [0.000 0.000 0.600 0.583 0.000 0.000]
 [1.000 1.000 1.000 1.000 1.000 0.667]]
```

Question 8

In this exercise we will implement a different type of data normalization.

a) Write a function `normal()` that normalizes a two-dimensional array (matrix) M such that the entries in each row have mean 0 and standard deviation 1. You can do this by subtracting each row's mean from every element in that row, and dividing every element by the standard deviation of the row. You may assume that the standard deviation of a row is not equal to zero.
Hint: A good understanding of the notion of broadcasting, see, e.g., Section 4.1, might be useful here.

It should give the following output on the given input M.

```
# Some test data
M = np.array([[ 1,  2,  3,  0],
       [ 5,  6,  -7,  0],
       [ -9, 10, 11, 0],
       [13, -13, 15, 0],
       [17, 18, 19, 0],
       [-21, -22, -23, 0]])
print(normal(M))
```

```
[[-0.447  0.447  1.342 -1.342]
 [ 0.777  0.971 -1.554 -0.194]
 [-1.472  0.858  0.981 -0.368]
 [ 0.822 -1.488  1.000 -0.333]
 [ 0.447  0.575  0.703 -1.725]
 [-0.471 -0.576 -0.680  1.727]]
```

b) Verify that the rows have mean 0 using the `np.mean()` function from NumPy.

c) Verify that the rows have standard deviation 1 using the `np.std()` function from NumPy.